

USB Device Security and BadUSB Attacks: A Comprehensive Analysis

Madeleine Callan

2025

Introduction

Universal Serial Bus (USB) technology has become the dominant interface for peripheral connectivity in modern computing systems. Since its introduction in 1996, the USB protocol has evolved rapidly, offering increasingly sophisticated functionality for an extensive range of devices to communicate with one another (USB Implementers Forum, 2014). While this evolution has dramatically improved compatibility and ease of use for consumers, it has simultaneously introduced significant security vulnerabilities that challenge traditional endpoint protection strategies.

The security implications of USB connectivity extend far beyond the commonly understood risks of malware-laden storage devices. A more insidious category of threats has emerged, exploiting fundamental design decisions in the USB specification itself. The BadUSB vulnerability, first publicly disclosed at the Black Hat USA 2014 security conference by researchers Karsten Nohl and Jakob Lell, demonstrated that the firmware residing in USB device controllers could be re-programmed to conceal attack code, taking advantage of a weakness common to the vast majority of USB peripheral devices: the absence of protection guaranteeing that any new firmware possesses the manufacturer’s unforgeable digital signature (Nohl & Lell, 2014).

The significance of BadUSB lies not merely in its exploitation potential, but in its fundamental challenge to the trust model underlying USB device interaction. Unlike conventional malware that operates at the software level and can theoretically be detected by antivirus solutions, BadUSB attacks operate at the firmware level, below the operating system’s visibility. As security researchers have noted, once infected, computers and their USB peripherals can never be fully trusted again without complete firmware verification (Blanchet, 2018). This paper provides a comprehensive examination of BadUSB attacks, analyzing the historical context that enabled their emergence, the technical

mechanisms underlying their operation, documented attack implementations, and the defensive countermeasures available to mitigate this persistent threat.

Background: The Evolution from PS/2 to USB

The PS/2 Interface

To fully appreciate the security implications introduced by USB technology, it is instructive to examine its predecessor: the PS/2 interface. IBM introduced the PS/2 port in 1987 alongside the Personal System/2 computer series, establishing what would become the standard keyboard and mouse interface for over a decade (IBM Corporation, 1987). The PS/2 interface employs a bidirectional synchronous serial protocol using a simple 6-pin mini-DIN connector, with dedicated ports color-coded purple for keyboards and green for mice according to the PC 97 specification.

The PS/2 keyboard interface is electrically identical to the 5-pin DIN connector used on earlier AT keyboards, differing only in physical connector form factor. This design implements direct digital I/O lines connecting the microcontroller in the external device to the microcontroller on the motherboard, creating a straightforward communication pathway (Chapweske, 2003). The protocol itself is remarkably simple: devices transmit data in frames consisting of 11-12 bits, including start bits, data bits, parity, and stop bits.

From a security perspective, the PS/2 interface possesses several characteristics that inherently limit its attack surface. The protocol is strictly defined for keyboard and mouse input; a device connected to a PS/2 port cannot masquerade as a different class of device. The interface is not hot-pluggable, requiring a system restart when devices are connected or disconnected. Most significantly, PS/2 devices generally lack reprogrammable firmware, relying instead on fixed microcontroller programming established during manufacturing. These constraints, while limiting flexibility, create an environment where the class of attacks enabled by BadUSB becomes fundamentally impossible.

The USB Paradigm Shift

The Universal Serial Bus specification, finalized in 1996, represented a fundamental departure from the dedicated-purpose design philosophy of PS/2. USB was conceived as a universal connectivity standard capable of supporting an extraordinarily diverse ecosystem of peripheral devices through a single standardized interface. This flexibility is achieved through a sophisticated enumeration process whereby devices identify their capabilities to the host system upon connection.

When a USB device is connected, it presents a device descriptor to the host containing information about its vendor, product, and supported device classes. The USB device class specification defines categories including Human Interface Devices (keyboards, mice), Mass Storage, Communication Devices, Audio, and numerous others. A single physical USB device may expose multiple interfaces across different classes, enabling compound devices that combine functionality—for example, a keyboard with integrated USB hub and storage capabilities.

This architectural flexibility, while enabling remarkable innovation in peripheral design, introduced security vulnerabilities not present in earlier interface standards. The device descriptor information is provided by the device itself with no cryptographic verification by the host system. The host operating system fundamentally trusts devices to accurately represent their capabilities, loading appropriate drivers based on the claimed device class. Furthermore, the microcontrollers embedded in USB devices frequently support firmware updates, a feature intended to enable manufacturers to correct defects and add capabilities but which creates the attack vector exploited by BadUSB.

The BadUSB Attack Model

Fundamental Mechanism

The BadUSB attack exploits the reprogrammable nature of firmware in USB device controller chips. The controller chip firmware, which governs how the device identifies itself and communicates with host systems, can in many devices be modified after manufacture. Security researchers demonstrated that by reverse-engineering the firmware update mechanisms of USB controller chips—particularly those manufactured by Phison Electronics, one of the largest USB controller suppliers—attackers could inject arbitrary code into devices previously considered trustworthy (Nohl & Lell, 2014).

The attack is particularly insidious because it occurs at a level below operating system visibility. When a compromised USB device is connected to a system, it can present itself as any device class the attacker chooses. A USB flash drive, for example, could simultaneously or sequentially present itself as a keyboard, enabling it to inject keystrokes as if a human were typing. Because the malicious code resides in firmware rather than in files on the storage medium, no amount of reformatting or file scanning will remediate the compromise. The malware persists through any action short of re-flashing the firmware with verified clean code—a procedure that is impractical for most users and often impossible without specialized equipment.

Attack Vectors and Capabilities

BadUSB-style attacks enable several categories of malicious activity. The most common exploitation pattern involves Human Interface Device (HID) emulation, where the compromised device identifies itself as a keyboard to the host system. Keyboards occupy a privileged position in the trust hierarchy of computing systems; they are automatically recognized without additional driver installation and possess the capability to input arbitrary commands. A malicious device emulating a keyboard can inject keystrokes that open command prompts, download and execute malware, modify system configurations, or exfiltrate data—all within seconds of connection.

Network adapter spoofing represents another powerful attack vector. A USB device presenting itself as an Ethernet adapter can intercept network traffic, modify DNS settings to redirect the victim’s web traffic through attacker-controlled servers, or establish persistent backdoor connections. This attack is particularly effective because operating systems typically auto-configure network interfaces without user intervention, and the additional network interface may operate undetected alongside legitimate connections.

The attack surface extends to boot-time exploitation as well. USB devices can be configured to interfere with the boot process, loading malicious code before the operating system’s security mechanisms are initialized. External storage devices or compromised USB hubs can inject boot sector viruses that establish persistent control before any defensive software loads (Security Research Labs, 2014).

Self-Propagation Capabilities

Perhaps most concerning is the potential for BadUSB malware to self-propagate. A compromised USB device connected to an infected host can, in turn, attempt to compromise other USB devices connected to the same system. If those devices also contain reprogrammable controllers with vulnerable firmware, the malware can spread from device to device without user awareness, creating a scenario where trusted devices become attack vectors in subsequent environments. This property transforms USB devices from passive attack tools into active propagation mechanisms, significantly amplifying the threat landscape.

Documented Attack Implementations

The USB Rubber Ducky

The USB Rubber Ducky, developed by security company Hak5, represents one of the earliest commercial implementations of keystroke injection attacks. While

not technically a BadUSB attack in the sense of reprogramming existing device firmware, the Rubber Ducky demonstrates the exploitation potential of HID emulation. The device appears to host systems as a standard USB keyboard but executes pre-programmed keystroke payloads at speeds exceeding human typing capability (Hak5, 2020).

The Rubber Ducky employs a simple scripting language called DuckyScript that allows security researchers and penetration testers to define keystroke sequences executed automatically upon device connection. Common payloads include reverse shell establishment, credential harvesting through simulated phishing interfaces, and system configuration modification. The device’s effectiveness in penetration testing scenarios has demonstrated that even security-conscious organizations often fail to implement adequate USB device controls.

The O.MG Cable

The O.MG Cable, demonstrated by security researcher Mike Grover, advances the concept of malicious USB hardware by embedding attack capabilities within what appears to be a standard USB charging and data cable (Grover, 2020). The cable contains a wireless implant that allows remote command and control, enabling an attacker to inject keystrokes, deploy payloads, or establish persistence at will after the cable is connected to a target system.

The implications of cable-based attacks are particularly severe for organizational security. Charging cables are ubiquitous in modern work environments, and users routinely connect phones and other devices using cables of unknown provenance. Unlike USB flash drives, which have received considerable security scrutiny, cables are generally perceived as passive components incapable of malicious behavior. The O.MG Cable demonstrates that this assumption is fundamentally incorrect, and that any component in the USB chain—including seemingly inert cables—may harbor malicious capabilities.

Flipper Zero and Modern HID Attacks

The Flipper Zero, a portable multi-tool for hardware security research, includes BadUSB functionality among its capabilities (Flipper Devices, 2021). The device can emulate USB HID devices and execute keystroke injection attacks, making techniques previously requiring specialized knowledge accessible to a broader audience. While marketed for legitimate security research and educational purposes, the accessibility of such tools underscores the democratization of USB-based attack capabilities.

The Spyduino project further demonstrates the accessibility of HID exploitation, using commonly available Arduino microcontrollers reprogrammed to appear as Human Interface Devices (Dalamagkidis, Ioannidis, & Antonatos, 2019). The project embeds the malicious Arduino within a USB keyboard, enabling it to

capture and exfiltrate sensitive information to cloud servers without user awareness. Such implementations require only modest technical expertise and minimal financial investment, lowering the barrier to entry for potential attackers.

Defensive Countermeasures

The Challenge of Detection

Traditional security mechanisms prove fundamentally inadequate against BadUSB attacks. Antivirus software operates at the file system level, scanning files stored on devices for known malware signatures. BadUSB malware resides in device firmware, a region inaccessible to conventional scanning tools. The operating system has no mechanism to verify that a device’s claimed identity matches its actual capabilities, and driver loading occurs automatically based on unverified device descriptors.

Even sophisticated endpoint detection and response (EDR) solutions face significant challenges. While behavioral analysis might detect the effects of a keystroke injection attack—such as rapid command execution or unusual process creation—the attack window may be sufficiently brief to accomplish its objectives before detection triggers response. The speed at which automated keystroke injection can execute malicious payloads often outpaces real-time detection capabilities.

USB Port Control and Physical Security

The most straightforward defensive measure involves physical control over USB ports. Organizations may implement policies requiring the physical disabling of unused USB ports through hardware blocks or BIOS configuration. Some security-sensitive environments employ PS/2 keyboards and mice specifically because the PS/2 interface cannot be exploited through BadUSB-style attacks, as the protocol is strictly limited to keyboard and mouse input and devices generally lack reprogrammable firmware (Cybersecurity and Infrastructure Security Agency, 2016).

However, physical port control is increasingly impractical in modern computing environments where USB connectivity is essential for legitimate business operations. The proliferation of USB-connected peripherals, charging requirements for mobile devices, and user expectations of convenient connectivity all work against restrictive physical controls.

Device Whitelisting with USBGuard

Software-based device whitelisting represents a more flexible approach to USB security. The USBGuard software framework, included in Red Hat Enterprise Linux and available for other Linux distributions, implements USB device authorization policies based on device attributes (Red Hat, 2019). When a USB device is connected, USBGuard evaluates it against a configured policy before allowing the device to interact with the system.

USBGuard creates device fingerprints based on multiple attributes including vendor and product identifiers, device name, serial number when available, interface types exposed, and the physical port to which the device connects. Administrators can generate initial policies based on currently connected trusted devices, then configure the system to block or require explicit authorization for any device not matching the whitelist.

The effectiveness of whitelisting approaches, however, is limited by fundamental weaknesses in USB device identification. Device descriptors can be spoofed, and if an attacker obtains knowledge of whitelisted device attributes, they can configure malicious hardware to match. Devices lacking serial numbers—common among inexpensive peripherals—cannot be uniquely identified, limiting whitelisting to device class and vendor information that is easily replicated. Nevertheless, whitelisting significantly raises the bar for casual attacks and provides logging capabilities for forensic investigation.

Interface and Behavior Restrictions

Beyond device-level whitelisting, security policies can restrict the specific interfaces a device is permitted to expose. A USB mass storage device claiming additional HID keyboard interfaces—a signature of certain BadUSB attacks—can be blocked based on the mismatch between expected and presented capabilities. GoodUSB, a research prototype, implements this approach by presenting users with a graphical interface to specify expected device functionality and rejecting any usage beyond the stated description (Tian, Bates, & Butler, 2015).

Temporal analysis of device behavior offers another detection avenue. Keystroke injection attacks exhibit characteristic timing patterns distinct from human typing. Malicious devices typically inject commands rapidly to minimize detection windows, while human keyboard input follows statistically predictable patterns with variable inter-key timing. The USBBlock system leverages this distinction to detect and block BadUSB-like attacks by correlating input signals with expected human behavior patterns (Neuner, Voyiatzis, Fotopoulos, Mulliner, & Weippl, 2018).

Firmware Integrity and Secure Updates

Addressing the root cause of BadUSB vulnerabilities requires fundamental changes to USB device firmware architecture. Firmware signing mechanisms, where devices will only accept firmware updates bearing cryptographic signatures from authorized sources, can prevent unauthorized modification. The National Institute of Standards and Technology (NIST) has published guidance on secure firmware update practices applicable to USB and other embedded devices (National Institute of Standards and Technology, 2018).

Trusted Platform Module (TPM) integration offers another architectural approach, enabling platforms to verify the integrity of connected devices through cryptographic attestation. The Trusted Computing Group has developed specifications for extending platform integrity verification to include peripheral firmware, though adoption remains limited (Trusted Computing Group, 2019).

The practical challenge with firmware-based solutions lies in the existing installed base of vulnerable devices. Billions of USB devices currently in use lack secure firmware update mechanisms and cannot be retroactively protected. Even for newly manufactured devices, implementing secure boot and firmware signing adds cost and complexity that may not be prioritized by manufacturers of low-margin peripherals.

The Evolving Threat Landscape

USB Type-C and New Attack Surfaces

The introduction of USB Type-C connectors and USB 3.x protocols has expanded both functionality and attack surface. The BADUSB-C attack model demonstrates that USB Type-C’s enhanced capabilities, including alternate display modes and power delivery negotiation, introduce new exploitation opportunities not present in earlier USB generations (Wang et al., 2021). The reversible connector design and increased bandwidth of Type-C enable attacks that were impractical with previous USB versions.

USB Power Delivery (USB-PD), the negotiated charging protocol used by Type-C devices, represents a particularly concerning attack vector. Malicious chargers or cables could potentially manipulate power negotiation to damage devices or exploit vulnerabilities in USB-PD implementations. The implicit trust placed in charging connections, combined with the complexity of modern USB protocols, creates opportunities for sophisticated attacks.

Social Engineering and USB Attacks

Technical countermeasures alone cannot fully address USB security risks because many attacks rely on social engineering to achieve initial access. Research has demonstrated that users will pick up and connect USB devices found in public locations, with some studies showing connection rates exceeding 45% for dropped USB drives (Tischer et al., 2016). Attackers can distribute compromised devices through various channels: leaving them in parking lots, mailing them to targets as promotional items, or substituting them for legitimate devices in supply chains.

The ByteBait simulation framework for BadUSB phishing campaigns demonstrates the continued effectiveness of social engineering approaches (Chen et al., 2025). As technical defenses improve, attackers increasingly rely on human factors to achieve device connection, underscoring the importance of user education alongside technical controls.

Conclusion

BadUSB represents a fundamental challenge to the security model underlying USB device connectivity. The vulnerability stems not from implementation errors that can be patched, but from architectural decisions that prioritized flexibility and ease of use over security verification. The trust that operating systems place in device self-identification, combined with the reprogrammable nature of USB controller firmware, creates an exploitation vector that operates below the visibility of conventional security tools.

The implications for organizational security are significant. USB ports, present on virtually every computing system, represent attack surfaces that cannot be fully secured through software means alone. The democratization of attack tools through products like the USB Rubber Ducky and Flipper Zero, combined with the availability of implementation guides for Arduino-based attacks, means that USB exploitation capabilities are accessible to attackers of modest technical sophistication.

Defensive strategies must be layered and appropriate to organizational risk tolerance. Device whitelisting through tools like USBGuard provides meaningful protection against casual attacks while maintaining operational flexibility. Behavioral analysis can detect certain attack patterns, and user education can reduce social engineering risks. However, organizations with stringent security requirements may find that physical port control or the use of legacy PS/2 interfaces for essential input devices remains the most reliable countermeasure.

Looking forward, the security of USB devices will depend on industry adoption of secure firmware practices, including cryptographic signing and integrity

verification mechanisms. Until such measures become universal, the fundamental trust assumption underlying USB connectivity—that devices are what they claim to be—will remain exploitable, and BadUSB will persist as a significant threat to endpoint security.

References

- Blanchet, S. (2018). BadUSB, the threat hidden in ordinary objects. *ResearchGate*. Retrieved from https://www.researchgate.net/publication/331876425_BadUSB_the_threat_hidden_in_ordinary_objects
- Chapweske, A. (2003). *The PS/2 keyboard interface*. Technical Documentation. Retrieved from https://www-ug.eecg.toronto.edu/msl/nios_devices/datasheets/PS2/%20Keyboard/%20Protocol.htm
- Chen, X. et al. (2025). ByteBait USB: A robust simulation toolkit for BadUSB phishing campaigns. *Journal of King Saud University - Computer and Information Sciences*. <https://doi.org/10.1007/s44443-025-00067-6>
- Cybersecurity and Infrastructure Security Agency. (2016). *USB device security: Guidance and mitigations*. CISA Advisory. Retrieved from <https://www.cisa.gov/>
- Dalamagkidis, K., Ioannidis, S., & Antonatos, S. (2019). Spyduino: Arduino as a HID exploiting the BadUSB vulnerability. *Proceedings of the 14th International Conference on Availability, Reliability and Security*. IEEE. <https://doi.org/10.1109/IC2IE47452.2019.8804730>
- Flipper Devices. (2021). *Flipper Zero: Portable multi-tool for hardware hacking*. Official Documentation. Retrieved from <https://flipperzero.one/>
- Grover, M. (2020). *OMG Cable—when cables attack*. Project Documentation. Retrieved from <https://omg.cable/>
- Hak5. (2020). *USB Rubber Ducky: Keystroke injection payload tool*. Product Documentation. Retrieved from <https://shop.hak5.org/products/usb-rubber-ducky>
- IBM Corporation. (1987). *IBM personal system/2 hardware interface technical reference*. IBM Corporation.
- National Institute of Standards and Technology. (2018). *Secure firmware update practices* [Special Publication]. NIST. Retrieved from NIST website: <https://www.nist.gov/>
- Neuner, S., Voyiatzis, A. G., Fotopoulos, S., Mulliner, C., & Weippl, E. R. (2018). USBlock: Blocking USB-based keypress injection attacks. *Proceedings of the 32nd Annual IFIP WG 11.3 Conference on Data and Applications Security and Privacy*, 278–295. Springer.
- Nohl, K., & Lell, J. (2014). BadUSB—on accessories that turn evil. *Black Hat USA 2014*. Las Vegas, NV. Retrieved from <https://www.blackhat.com/us-14/briefings.html>
- Red Hat. (2019). *Security guide: Using USBGuard*. Red Hat Enterprise Linux 7 Documentation. Retrieved from <https://docs.redhat.com/en/>

- documentation/red_hat_enterprise_linux/7/html/security_guide/sec-using-usbguard
- Security Research Labs. (2014). *BadUSB: On accessories that turn evil*. SRLabs Research. Retrieved from <https://srlabs.de/badusb/>
- Tian, D. J., Bates, A., & Butler, K. (2015). Defending against malicious USB firmware with GoodUSB. *Proceedings of the 31st Annual Computer Security Applications Conference*, 261–270. ACM.
- Tischer, M., Durumeric, Z., Foster, S., Duan, S., Mori, A., Bursztein, E., & Bailey, M. (2016). Users really do plug in USB drives they find. *Proceedings of the 2016 IEEE Symposium on Security and Privacy*, 306–319. IEEE.
- Trusted Computing Group. (2019). *TCG PC client platform firmware integrity measurement*. TCG Specification. Retrieved from <https://trustedcomputinggroup.org/>
- USB Implementers Forum. (2014). *Universal Serial Bus Specification*. USB-IF Official Specification. Retrieved from <https://www.usb.org/document-library>
- Wang, Z. et al. (2021). BADUSB-C: Revisiting BadUSB with type-c. *Proceedings of the 15th USENIX Workshop on Offensive Technologies (WOOT '21)*. USENIX Association. Retrieved from <https://fengweiz.github.io/paper/badusbc-woot21.pdf>